

"cadabra" parser on the remaining characters. So in order to support sequencing, we require a way of letting a Parser indicate how many characters it consumed. Capturing this is pretty easy:¹⁶

```
type Parser[+A] = Location => Result[A]
```

A parser now returns a **Result** that's either a success or a failure.

```
trait Result[+A]
```

```
case class Success[+A](get: A, charsConsumed: Int) extends Result[A]
```

```
case class Failure(get: ParseError) extends Result[Nothing]
```

In the success case, we return the number of characters consumed by the parser.

We introduced a new type here, `Result`, rather than just using `Either`. In the event of success, we return a value of type `A` as well as the number of characters of input consumed, which the caller can use to update the `Location` state.¹⁷ This type is starting to get at the essence of what a `Parser` is—it's a kind of state action that can fail, similar to what we built in chapter 6. It receives an input state, and if successful, returns a value as well as enough information to control how the state should be updated.

This understanding—that a `Parser` is just a state action—gives us a way of framing a representation that supports all the fancy combinators and laws we've stipulated. We simply consider what each primitive requires our state type to track (just a `Location` may not be sufficient), and work through the details of how each combinator transforms this state.



EXERCISE 9.13

Implement `string`, `regex`, `succeed`, and `slice` for this initial representation of `Parser`. Note that `slice` is less efficient than it could be, since it must still construct a value only to discard it. We'll return to this later.

9.6.3 Labeling parsers

Moving down our list of primitives, let's look at `scope` next. In the event of failure, we want to push a new message onto the `ParseError` stack. Let's introduce a helper function for this on `ParseError`. We'll call it `push`.¹⁸

```
def push(loc: Location, msg: String): ParseError =  
  copy(stack = (loc,msg) :: stack)
```

¹⁶ Recall that `Location` contains the full input string and an offset into this string.

¹⁷ Note that returning an `(A, Location)` would give `Parser` the ability to change the input stored in the `Location`. That's granting it too much power!

¹⁸ The `copy` method comes for free with any `case class`. It returns a copy of the object, but with one or more attributes modified. If no new value is specified for a field, it will have the same value as in the original object. Behind the scenes, this just uses the ordinary mechanism for default arguments in Scala.